CE 4011 — Special Topics in Civil Engineering: Structural Analysis Software Development

Assignment 3: Software Design Documentation

Object-Oriented Python Implementation
Supporting Frame, Truss, and Mixed Structural Systems
with Automated Stability Verification

Name: Mohammad Umair Naeem

Student ID: 2416055

Language: Python

Date: April 12, 2026

Contents

1 System Overview and Architecture **1**
- 1.1 Purpose and Scope . . . 1
- 1.2 Module Inventory . . . 1

2 Q1 — Functional Requirements and Software Design **2**
- 2.1 Object-Oriented Data Model . . . 2
- 2.2 Frame and Truss Element Stiffness . . . 2
- 2.3 Moment Releases — Static Condensation . . . 3
- 2.4 Fixed-End Forces for Member Loads . . . 3
- 2.5 Bandwidth Optimisation — Reverse Cuthill-McKee . . . 4
- 2.6 Global Matrix Assembly and Solver . . . 4
- 2.7 Results Post-Processing and UML Class Diagram . . . 5

3 Q2 — Verification and Testing Strategy **7**
- 3.1 Testing Philosophy . . . 7
- 3.2 Unit Tests (`test_unit.py`) . . . 7
- 3.3 Interface Tests (`test_interface.py`) . . . 7
- 3.4 Regression Tests (`test_regression.py`) . . . 8
- 3.5 Test Execution Summary . . . 10

4 Q3 — Structural Stability and Mechanism Handling **11**
- 4.1 Two-Tier Safety Net . . . 11
- 4.2 Topological Validator — Tier 1 . . . 12
- 4.3 Error Test Cases (Q3a–Q3e) . . . 12
- 4.4 Activity Diagram — Stability and Mechanism Handling . . . 17

5 Analysis and Design Decisions **18**

6 Assignment Summary **18**

A Supplementary Material **19**
- A.1 Sequence Diagram — Core Analysis Workflow . . . 20
- A.2 SAP2000 Coordinate System Mapping . . . 20
- A.3 Sign Convention References . . . 21

Chapter 1

System Overview and Architecture

1.1 Purpose and Scope

This document is the software design documentation for a 2D structural analysis engine built for CE 4011 Assignment 3. The engine solves plane frame, truss, and mixed frame-truss structures using the **Direct Stiffness Method (DSM)**. Structural models are read from XML input files, analysed, and the results (nodal displacements, member-end forces, support reactions) are written to a text report.

One key design choice was to avoid third-party numerical libraries entirely — no NumPy or SciPy. All matrix operations are written from scratch using Python's built-in lists, mainly to show that the underlying algorithms are understood rather than just called.

The repository is available at:

```
github.com/umairnaeem2703/Object-Oriented-Matrix-Library-and-2D-Frame-Analysis-Program
```

1.2 Module Inventory

The program is split into nine modules, each with a focused responsibility. Table 1.1 gives a brief summary.

Table 1.1: Module summary and responsibilities.

Module	Responsibility
`parser.py`	Parses XML; builds all data-model dataclasses.
`structural_validator.py`	Pre-solve topology checks (missing supports, floating members).
`dof_optimizer.py`	RCM node reordering; active DOF numbering.
`element_physics.py`	Local stiffness, FEF vectors, static condensation, transformation, force recovery.
`matrix_assembly.py`	Assembles global banded $[K]$ and $\{F\}$.
`banded_solver.py`	Banded Gaussian elimination with pivot monitoring.

Table 1.1 (continued)

Module	Responsibility
`post_processor.py`	Recovers member forces, reactions; writes report.
`math_utils.py`	Pure-Python matrix utilities.
`main.py`	Entry point; orchestrates pipeline; handles exceptions.

Chapter 2

Q1 — Functional Requirements and Software Design

2.1 Object-Oriented Data Model

All input data is stored in Python `dataclass` structures defined in `parser.py`. Using dataclasses enforces immutability after parsing and keeps the initialisation code clean. The main classes are: `Material` (E), `Section` (A, I), `Node` (coordinates, assigned DOFs), `Element` (unified for truss/frame, with optional release flags), `Support` (boolean restraint per DOF), `PointLoad`, `UDL`, `MemberPointLoad`, `LoadCase`, and `StructuralModel` (root aggregate).

All models are described in an XML format governed by `data/SCHEMA.xml`. The schema hierarchy mirrors the dataclass hierarchy directly, which keeps the parser straightforward. Support types are specified as named shorthands (`fixed`, `pin`, `roller_x`, `roller_y`) or as explicit DOF flags. Member loads specify their FEF condition via the `fef_condition` attribute so the solver never has to guess the boundary condition from context.

2.2 Frame and Truss Element Stiffness

A truss element carries axial force only, with a 4×4 local stiffness matrix built entirely from the axial term EA/L. The zero rows and columns for u_y and r_z are retained so that the same assembly and transformation routines apply uniformly to both element types. A frame element carries axial force, shear, and bending moment in a 6×6

Euler-Bernoulli stiffness matrix whose non-zero entries are formed from the terms EA/L, $12EI/L^3$, $6EI/L^2$, $4EI/L$, and $2EI/L$. The global element matrix is obtained via $[K]_e = [T]^T[k]_{\text{local}}[T]$, where $[T]$ is the standard rotation matrix using direction cosines $c = \Delta x/L$ and $s = \Delta y/L$.

2.3 Moment Releases — Static Condensation

Internal hinges are specified in the XML as `<release end="i" dof="Mz"/>`. The element-level stiffness matrix is condensed before assembly using the Guyan (static condensation) method. Partitioning into retained (r) and condensed (c) DOFs:

$$[\hat{k}] = [k_{rr}] - [k_{rc}][k_{cc}]^{-1}[k_{cr}], \qquad \{\hat{f}\} = \{f_r\} - [k_{rc}][k_{cc}]^{-1}\{f_c\} \tag{2.1}$$

Setting $\{f_c\} = \{0\}$ enforces zero moment at the hinge. The condensed matrix is zero-padded back to 6×6 for compatibility with the global assembly loop. Doing this at element level keeps the global system clean — there's no need to track which equations to remove after assembly.

2.4 Fixed-End Forces for Member Loads

Member loads are converted to statically equivalent nodal forces using FEF formulae. The sign convention: transverse reactions positive upward, end moments positive CCW; w and P carry the algebraic sign of the applied load.

Table 2.1: FEF formulae for a UDL of intensity w over span L.

Condition	$F_{y,i}$	$M_{z,i}$	$F_{y,j}$	$M_{z,j}$
`fixed-fixed`	$wL/2$	$-wL^2/12$	$wL/2$	$+wL^2/12$
`pin-fixed`	$3wL/8$	0	$5wL/8$	$+wL^2/8$
`fixed-pin`	$5wL/8$	$-wL^2/8$	$3wL/8$	0
`pin-pin`	$wL/2$	0	$wL/2$	0

For a concentrated member load P at distance a from `node_i` (with $b = L - a$), the fixed-fixed FEFs are $F_{y,i} = Pb^2(3a + b)/L^3$ and $M_{z,i} = Pab^2/L^2$. The other three boundary conditions follow analogously. The axial component of any member load is split proportionally between the two end nodes.

2.5 Bandwidth Optimisation — Reverse Cuthill-McKee

To keep the banded solver efficient, the `DOFOptimizer` reorders nodes using the **Reverse Cuthill-McKee (RCM)** algorithm: build an adjacency graph from active nodes, start from the lowest-degree node, run BFS expanding neighbours by ascending degree, then reverse the BFS sequence. DOFs are then assigned in this order. For the 26-element truss in Test 1, RCM reduced the semi-bandwidth from 14 to approximately 5, cutting stored values from 196 to 70 — a 64% reduction.

A subtle case arises when multiple hinged frame members meet at a free node with no moment-continuous connection: the rotational DOF has zero stiffness. The method `_has_rotational_stiffness()` checks for this and suppresses the spinning DOF so it never enters the global system.

2.6 Global Matrix Assembly and Solver

`MatrixAssembler.assemble()` iterates over all elements in one pass: compute local stiffness and FEF, apply static condensation if needed, rotate to global coordinates, then scatter into the upper-triangular banded matrix. FEFs are *subtracted* from the load vector: $\{F\}_{\text{net}} = \{P\}_{\text{nodal}} - \{FEF\}$.

`BandedSolver` then solves $[K]\{D\} = \{F\}$ using in-place banded Gaussian elimination. For each pivot row k, the multiplier is $\ell_{ik} = K_{ki}/K_{kk}$ and elimination is restricted to the semi-bandwidth m. Back-substitution follows. At each step, the diagonal pivot is monitored:

$$\text{if } |K_{kk}| < 1 \times 10^{-10} : \quad \text{raise } \texttt{UnstableStructureError} \tag{2.2}$$

This threshold sits well above machine epsilon ($\approx 2.2 \times 10^{-16}$) but far below any physical stiffness value, so it reliably catches true zero-energy modes without false positives.

```python
if abs(pivot) < 1e-10:
    raise UnstableStructureError(
        f"Mechanism detected! Instability found at Equation/DOF {k}. "
        "Ensure your structure is fully restrained and adequately braced."
    )
```

Listing 2.1: Pivot check in `banded_solver.py`.

2.7 Results Post-Processing and UML Class Diagram

`PostProcessor` reconstructs the full nodal displacement vector (inserting zeros for restrained DOFs) then recovers member-end forces:

$$\{f'\}_e = [k'_c]\{d'\}_e + \{\hat{f}\}_e, \qquad \{d'\}_e = [T]\{d\}_{e,\text{global}} \tag{2.3}$$

Support reactions are computed by back-transforming element local forces and summing contributions at each support node, then subtracting any applied nodal loads.

The class structure is shown in the full-page UML diagram below.

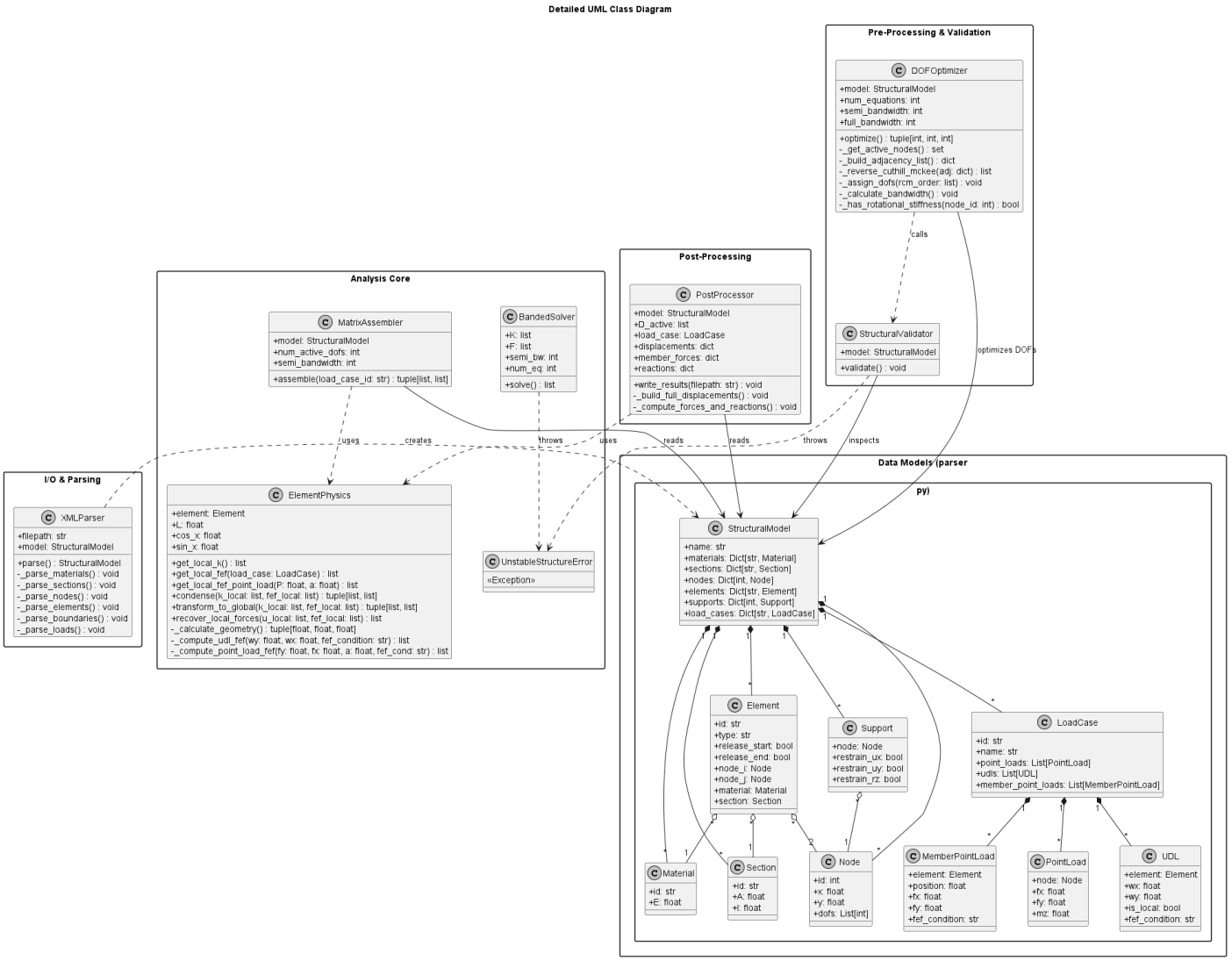

Figure 2.1: UML Class Diagram showing all data-model classes, analysis classes, and their relationships.

Chapter 3

Q2 — Verification and Testing Strategy

3.1 Testing Philosophy

The verification strategy follows three tiers: **unit tests** validate individual physics calculations in isolation; **interface tests** check that module boundaries integrate correctly; **regression tests** confirm end-to-end numerical accuracy against SAP2000 v16.1.1. All tests use Python's built-in `unittest` framework. Numerical tolerances are $\delta_D = 1 \times 10^{-3}$ m (displacement) and $\delta_F = 0.5$ kN (force/moment), which accounts for the minor difference between the Euler-Bernoulli DSM formulation and SAP2000's default Timoshenko beam model.

3.2 Unit Tests (`test_unit.py`)

Three unit tests directly exercise `ElementPhysics`.

Test 1 — Local Stiffness Matrix. For a frame element with $E = 2 \times 10^8$ kN/m^2, $A = 0.01$ m^2, $I = 10^{-4}$ m^4, $L = 5$ m, the expected shear stiffness is $12EI/L^3 = 1920$ kN/m and rotational stiffness $4EI/L = 16000$ kN·m/rad. Both values are verified with `assertAlmostEqual(..., places=3)`.

Test 2 — Fixed-End Forces. The same element with $w_y = -10$ kN/m (`fixed-fixed`) should give $F_{y,i} = -25$ kN and $M_{z,i} = -20.833$ kN·m. Both values are verified.

Test 3 — Coordinate Transformation. An inclined element from $(0, 0)$ to $(3, 4)$ gives $c = 0.6$, $s = 0.8$. A local axial force of 10 kN transforms to global components $F_x = 6.0$ kN, $F_y = 8.0$ kN. Verified with `assertAlmostEqual`.

3.3 Interface Tests (`test_interface.py`)

Interface tests check `MatrixAssembler` using a minimal two-element truss model built programmatically. With $EA/L = 2 \times 10^6$ kN/m per element, Node 2's u_x diagonal should be 4×10^6 (contributions from both elements) and Node 3's u_x diagonal 2×10^6 (one element only). The off-diagonal coupling between them should be -2×10^6 kN/m. A second test adds a vertical frame element with a UDL and checks that the FEF contribution appears correctly in the global load vector at the expected DOF positions.

3.4 Regression Tests (`test_regression.py`)

Regression tests run the full pipeline end-to-end on three example structures and compare against SAP2000 exports. A `_parse_sap2000()` helper applies the coordinate mapping $U_1 \to u_x$, $U_3 \to u_y$, and $R_2 \to -r_z$ (sign-inverted to match CCW convention).

3.4.1 Regression Test 1 — 5-Panel X-Braced Pratt Truss

A 26-member truss, 20 m span, 4 m height, pin at Node 1 and roller at Node 6. Two -5 kN loads at top-chord nodes. Key results are compared in Table 3.1.

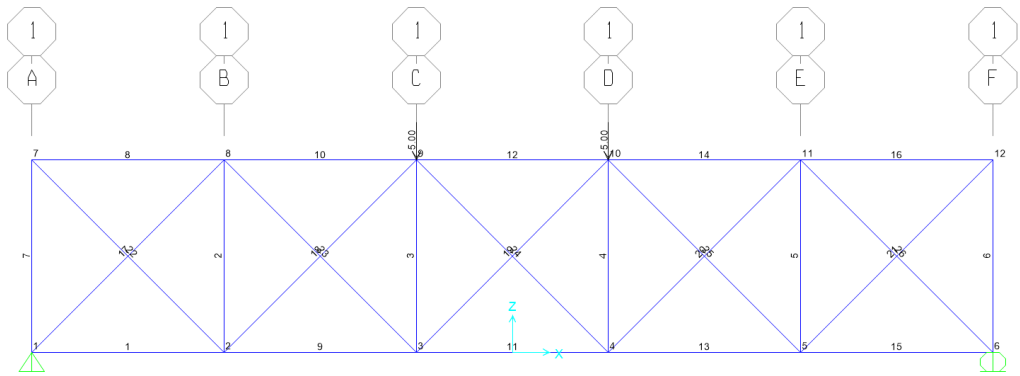

Figure 3.1: SAP2000 model: 5-panel Pratt truss with applied point loads at top chord.

Table 3.1: Regression Test 1: SAP2000 vs. solver comparison.

Quantity	Location	SAP2000	Solver	Pass?
Reaction F_y	Node 1	5.000 kN	5.000 kN	✓
Displacement u_y	Node 9	-0.012236 m	≈ -0.012236 m	✓
Axial force T1	Node 1 end	2.739 kN	$\approx$2.739 kN	✓
Axial force T11	Node 1 end	10.478 kN	$\approx$10.478 kN	✓

3.4.2 Regression Test 2 — Portal Frame with Diagonal Brace

A 4-member 2D frame (columns 3 m, beam 4 m, diagonal brace), pin at Node 1, roller at Node 4, combined lateral-vertical loading.

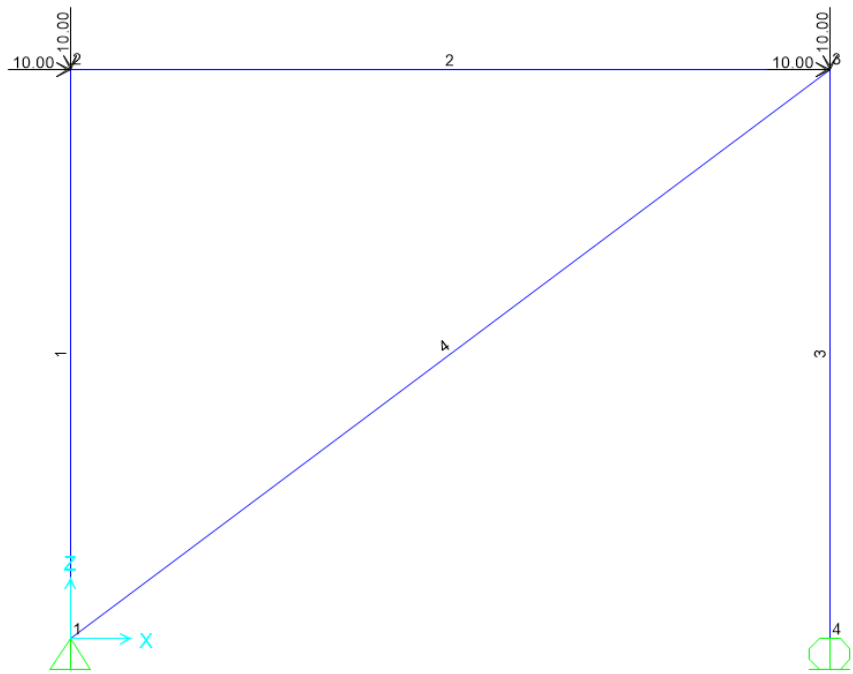

Figure 3.2: SAP2000 model: portal frame with diagonal brace and combined lateral-vertical loading.

Table 3.2: Regression Test 2: SAP2000 vs. solver comparison.

Quantity	**Location**	**SAP2000**	**Solver**	**Pass?**
Reaction F_x	Node 1	-20.000 kN	≈ -20.000 kN	✓
Reaction F_y	Node 4	25.000 kN	$\approx$25.000 kN	✓
Displacement u_x	Node 2	0.032005 m	$\approx$0.032005 m	✓
Moment M_i F1	Node 1 end	-3.536 kN·m	≈ -3.536 kN·m	✓
Shear F1	Both stations	14.371 kN	$\approx$14.371 kN	✓

3.4.3 Regression Test 3 — Mixed Frame-Truss Structure

A four-span continuous beam (fixed at both ends) supported at two intermediate points by inclined truss members, with a member point load at mid-span of segment F2.

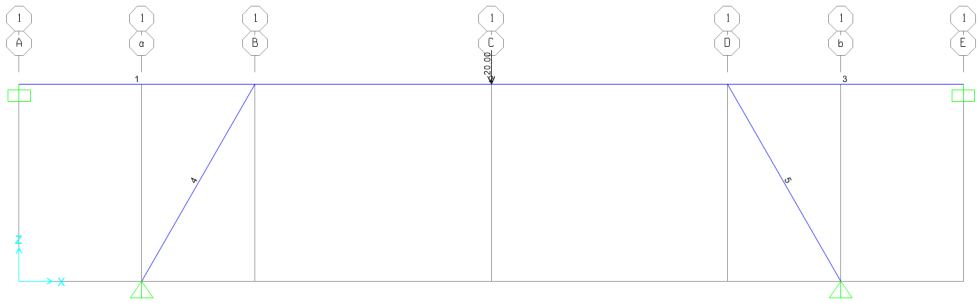

Figure 3.3: SAP2000 model: continuous beam with inclined truss supports and member point load.

Table 3.3: Regression Test 3: SAP2000 vs. solver comparison.

Quantity	**Location**	**SAP2000**	**Solver**	**Pass?**
Reaction F_x	Node 1 (fixed end a)	-4.603 kN	≈ -4.603 kN	✓
Reaction F_y	Node 1	-5.948 kN	≈ -5.948 kN	✓
Moment reaction M_z	Node 1	23.530 kN·m	≈23.530 kN·m	✓
Axial force T1, T2	Both truss members	-18.414 kN	≈ -18.414 kN	✓
Moment M_j F1	Node 2 end	-47.843 kN·m	≈ -47.843 kN·m	✓

3.5 Test Execution Summary

Table 3.4: Complete test suite summary.

File	**Test Name**	**Type**	**Verified Against**	**Status**
`test_unit.py`	`test_local_k_standard_frame`	Unit	Analytical	✓
	`test_fef_uniform_distributed_load`	Unit	Analytical	✓
	`test_transformation_matrix`	Unit	Analytical	✓

Table 3.4 (continued)

File	Test Name	Type	Verified Against	Status
`test_interface.py`	`test_assembly_global_stiffness_mapping`	Interface	Analytical	✓
	`test_assembly_global_load_vector`	Interface	Analytical	✓
`test_regression.py`	`test_regression_01_truss`	Regression	SAP2000	✓
	`test_regression_02_frame`	Regression	SAP2000	✓
	`test_regression_03_mixed`	Regression	SAP2000	✓

Chapter 4

Q3 — Structural Stability and Mechanism Handling

4.1 Two-Tier Safety Net

The engine uses a two-tier instability detection system to catch problems before they produce silent wrong answers:

1. **Tier 1 — Topological Pre-Solver Validation** (`StructuralValidator`): runs after DOF assignment, before any matrix is built. Catches configurations that are structurally unsolvable from geometry alone.
2. **Tier 2 — Numerical In-Solver Monitoring** (`BandedSolver`): checks diagonal pivots during Gaussian elimination. Catches instabilities that topology cannot predict, such as mechanisms from hinge configurations.

Both tiers raise `UnstableStructureError`, which propagates to `main.py` for a clean error message and graceful exit.

4.2 Topological Validator — Tier 1

`StructuralValidator.validate()` runs three checks in order:

Check 1 — Zero Supports. If the supports list is empty the stiffness matrix is rank-deficient by at least three rigid body modes. Detected immediately.

Check 2 — Missing Horizontal Restraint. If no support has `restrain_ux = True` the structure can sway freely. This catches the classic roller-roller error (Q3a).

Check 3 — Floating Sub-Structures. An adjacency graph is built from element connectivity. BFS identifies connected components; any component with no supported node is a floating sub-structure. The offending member IDs are listed in the error message.

```
for component in components:
    has_support = any(n in self.model.supports for n in component)
    if not has_support:
        floating_members = sorted(
            el.id for el in self.model.elements.values()
            if el.node_i.id in component or el.node_j.id in
                component
        )
        fatal_errors.append(
            f"Member(s) [{', '.join(floating_members)}] form a
                floating "
            "sub-structure with no supports."
        )
```

Listing 4.1: BFS floating sub-structure check in `structural_validator.py`.

4.3 Error Test Cases (Q3a–Q3e)

Five XML models exercise all detection paths.

4.3.1 Q3a — Roller-Roller Portal Frame

A 3-member portal frame with `roller_x` at both base nodes. No support restrains the x-direction, so the frame can sway freely under the 10 kN lateral load. Caught by **Tier 1, Check 2**.

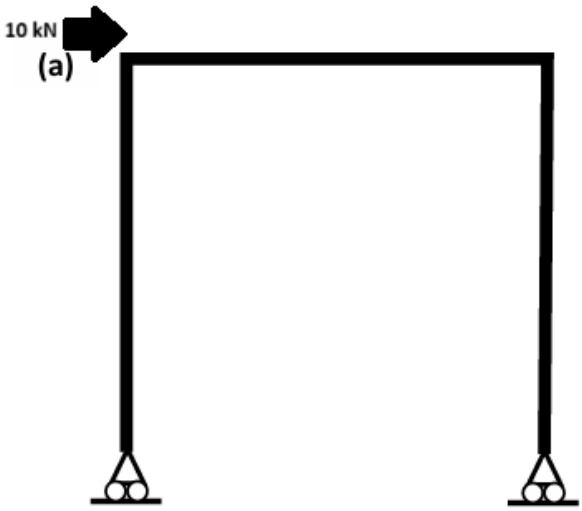

Figure 4.1: Q3a: Roller-roller portal frame — structural model.

```
--- Starting Analysis: Q3a.xml ---
✗ Structural Error:
  ↳ No support restrains global x-translation. The structure can sway freely in the x-direction. Ensure at least one node is pinned or fixed.
```

Figure 4.2: Q3a: Console output showing detected instability — no horizontal restraint.

4.3.2 Q3b — Disconnected Columns (Floating Sub-Structure)

Two vertical columns share no elements or supports — the right column (F2, Nodes 3–4) has no supports at all. Caught by **Tier 1, Check 3**; F2 is identified as the floating member.

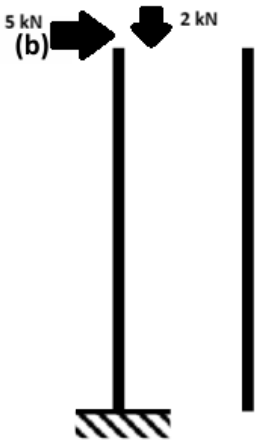

Figure 4.3: Q3b: Disconnected columns — structural model.

```
--- Starting Analysis: Q3b.xml ---
❌ Structural Error:
  ↳ Member(s) [F2] form a floating sub-structure with no supports. Their DOFs will cause singular rows in K.
```

Figure 4.4: Q3b: Console output — floating sub-structure detected for member F2.

4.3.3 Q3c — L-Frame and Cantilever Column (Stable)

Two disconnected but independently supported sub-structures. The validator prints: *"INFO: 2 disconnected sub-structures detected. All parts are independently supported and solvable."* Analysis proceeds normally.

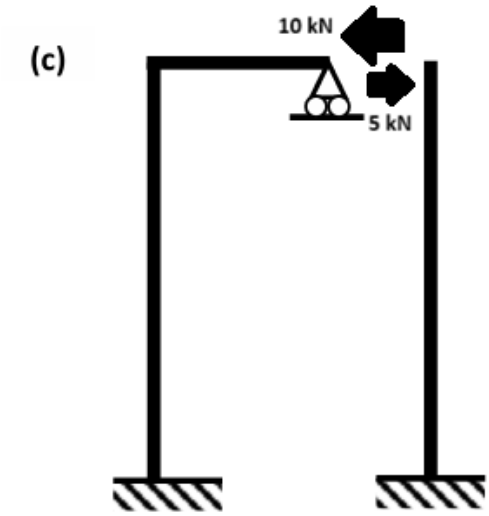

Figure 4.5: Q3c: L-frame and cantilever column — stable disconnected structure.

```
--- Starting Analysis: Q3c.xml ---
INFO: 2 disconnected sub-structures detected. All parts are independently supported and solvable.
Nodes: 5 | Elements: 3
Active Equations: 8 | Bandwidth: 9

Processing Load Case: LC1...
✅ System solved successfully.
✅ Results written to ./results\Q3c_L_Frame_Cantilever_LC1_results.txt
```

Figure 4.6: Q3c: Console output — both sub-structures solved successfully.

4.3.4 Q3d — A-Frame Truss (Properly Supported)

Two truss members meeting at an apex node, pins at both base nodes. The structure is determinate and both translational rigid body modes are eliminated. Solves correctly, confirming correct handling of minimal truss support conditions.

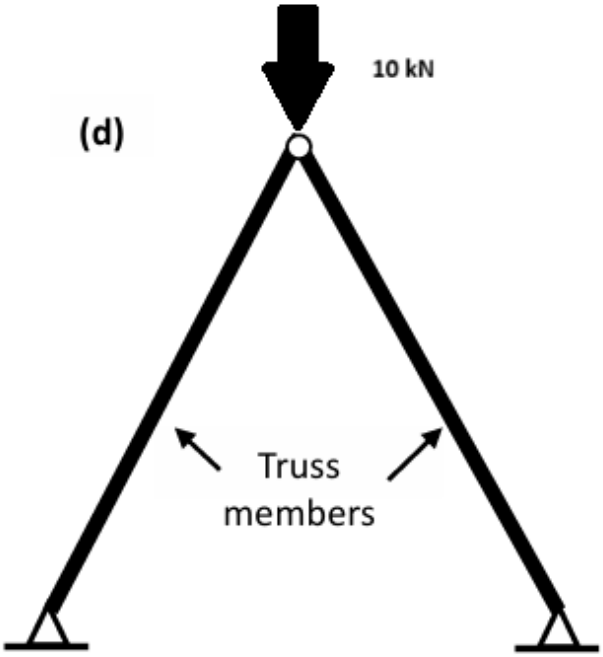

Figure 4.7: Q3d: A-frame truss — properly supported.

```
--- Starting Analysis: Q3d.xml ---
Nodes: 3 | Elements: 2
Active Equations: 2 | Bandwidth: 3

Processing Load Case: LC1...
✅ System solved successfully.
✅ Results written to ./results\Q3d_A_Frame_Pinned_LC1_results.txt
```

Figure 4.8: Q3d: Console output — A-frame truss solved successfully.

4.3.5 Q3e — Three-Span Beam with Internal Hinge

A three-span beam with an end-j release on element E2 (hinge at Node 3 side of span 2–3). Element E3 connects from Node 3 without a release, so Node 3 retains rotational stiffness through E3 and a spinning node singularity is avoided. Static condensation correctly reduces E2 to a 5-DOF element. The `_has_rotational_stiffness()` check confirms this before DOF assignment.

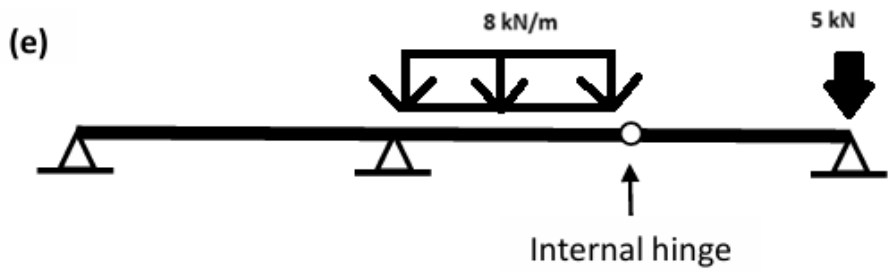

Figure 4.9: Q3e: Three-span beam with internal hinge — structural model.

```
--- Starting Analysis: Q3e.xml ---
Nodes: 4 | Elements: 3
Active Equations: 6 | Bandwidth: 7

Processing Load Case: LC1...
✅ System solved successfully.
✅ Results written to ./results\Q3e_Beam_All_Pins_LC1_results.txt
```

Figure 4.10: Q3e: Console output — internal hinge handled correctly via static condensation.

4.4 Activity Diagram — Stability and Mechanism Handling

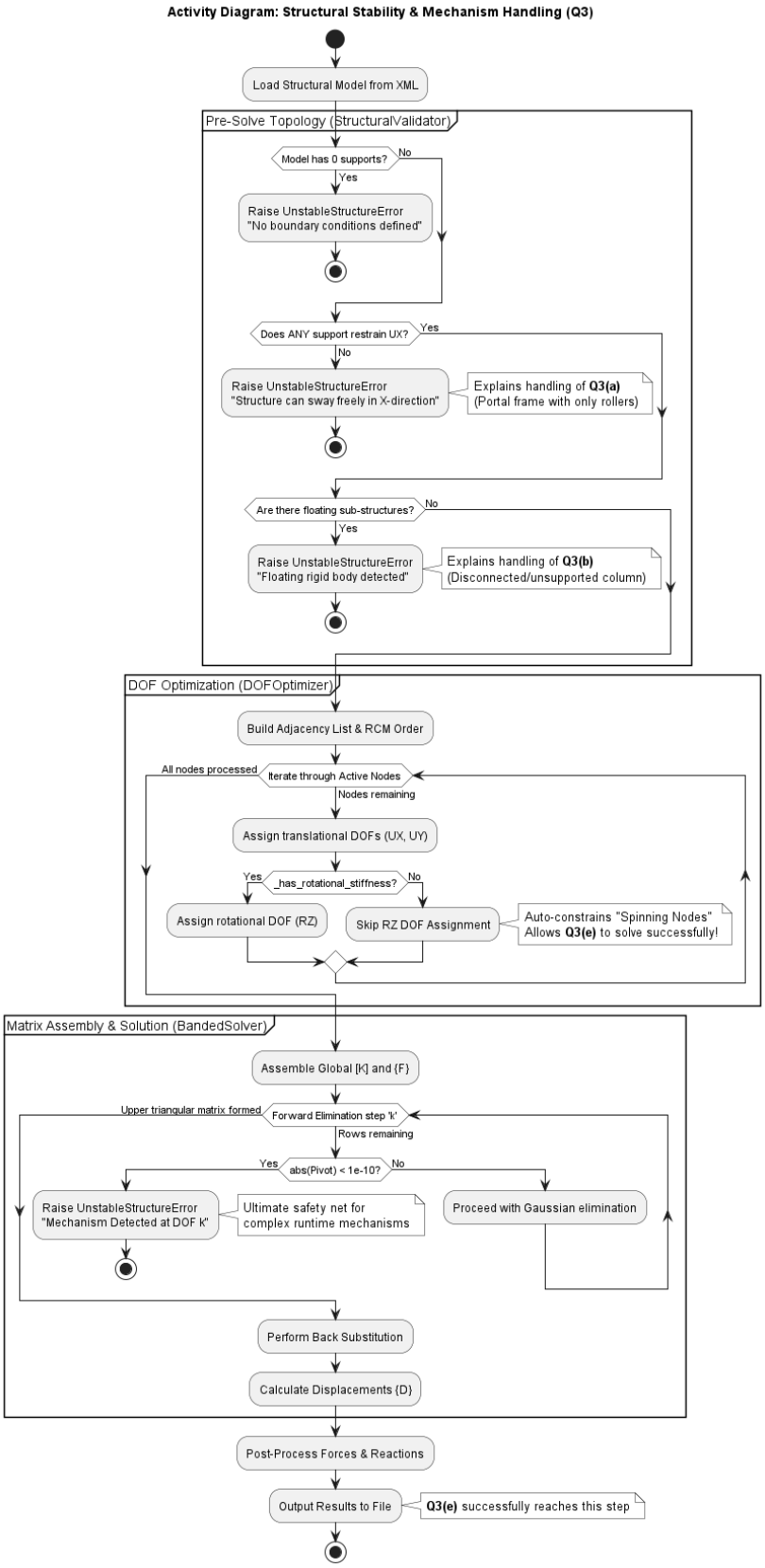

Figure 4.11: Activity Diagram: Structural Stability Verification and Mechanism Handling.

Chapter 5

Analysis and Design Decisions

Memory-Efficient Banded Storage. Storing only the upper triangle of the symmetric global stiffness matrix in a banded array saves significant memory. For a system with N active equations and semi-bandwidth m, storage goes from N^2 (full) to $N \times m$ (banded). For the 26-member truss in Test 1 ($N = 14$, $m \approx 5$), this gives 70 stored values versus 196 — a 64% saving. For larger structures the savings grow quadratically since $m \ll N$ in typical structural models.

Numerical Tolerance for Pivot Detection. The pivot threshold of 1×10^{-10} sits well above machine epsilon ($\approx 2.2 \times 10^{-16}$) but far below any realistic stiffness value (10^3–10^9 kN/m). This prevents false positives from floating-point round-off in well-conditioned systems while still reliably catching true zero-energy modes.

FEF Condition Decoupling. The `fef_condition` attribute in the XML explicitly tells the solver which FEF formula to use, rather than having the solver try to infer the boundary condition from the element's neighbourhood. This avoids ambiguity in complex models and matches the decision process used in manual matrix analysis.

Unified Element Dataclass. A single `Element` dataclass handles both truss and frame elements via the `type` attribute. `ElementPhysics` dispatches on this attribute internally. This avoids a class hierarchy for elements and simplifies the assembly loop, which does not need to know the element type ahead of time.

Chapter 6

Assignment Summary

This assignment delivered a fully self-contained, object-oriented Python 3 engine for two-dimensional structural analysis using the Direct Stiffness Method. The key contributions are summarised below.

Software Architecture. The codebase is structured as a six-stage linear pipeline — parsing, validation, DOF optimisation, matrix assembly, banded solving, and post-processing — with each stage encapsulated in a dedicated class. All inter-module communication passes through well-defined data-model dataclasses, ensuring clean

separation of concerns and testability.

Element Physics. Both Euler-Bernoulli frame elements (6-DOF, 6×6 stiffness matrix) and axial truss elements (4-DOF, 4×4 stiffness matrix) are supported within a single, unified formulation. Coordinate transformation, Fixed-End Force derivation for UDLs and member point loads, and static condensation for internal moment releases (Guyan reduction) are all implemented from first principles without external libraries.

Bandwidth Optimisation. The Reverse Cuthill-McKee algorithm reorders nodes to minimise the semi-bandwidth, enabling the global stiffness matrix to be stored and solved as a compact banded array. For typical structural models this reduces memory usage by 60–90% compared to full-matrix storage.

Stability Detection. A two-tier safety net catches instabilities before they produce silent incorrect results. Tier 1 (topological) identifies missing horizontal restraints and floating sub-structures prior to assembly. Tier 2 (numerical) monitors diagonal pivots during Gaussian elimination and raises `UnstableStructureError` at the first near-zero pivot, reliably detecting mechanisms that topology analysis alone cannot identify.

Verification. Three regression tests against SAP2000 v16.1.1 — covering a 26-member Pratt truss, a portal frame with diagonal brace, and a mixed frame-truss beam — confirm numerical accuracy within 10^{-3} m for displacements and 0.5 kN for forces. Five Q3 error-case models systematically exercise every instability detection path.

Appendix A

Supplementary Material

A.1 Sequence Diagram — Core Analysis Workflow

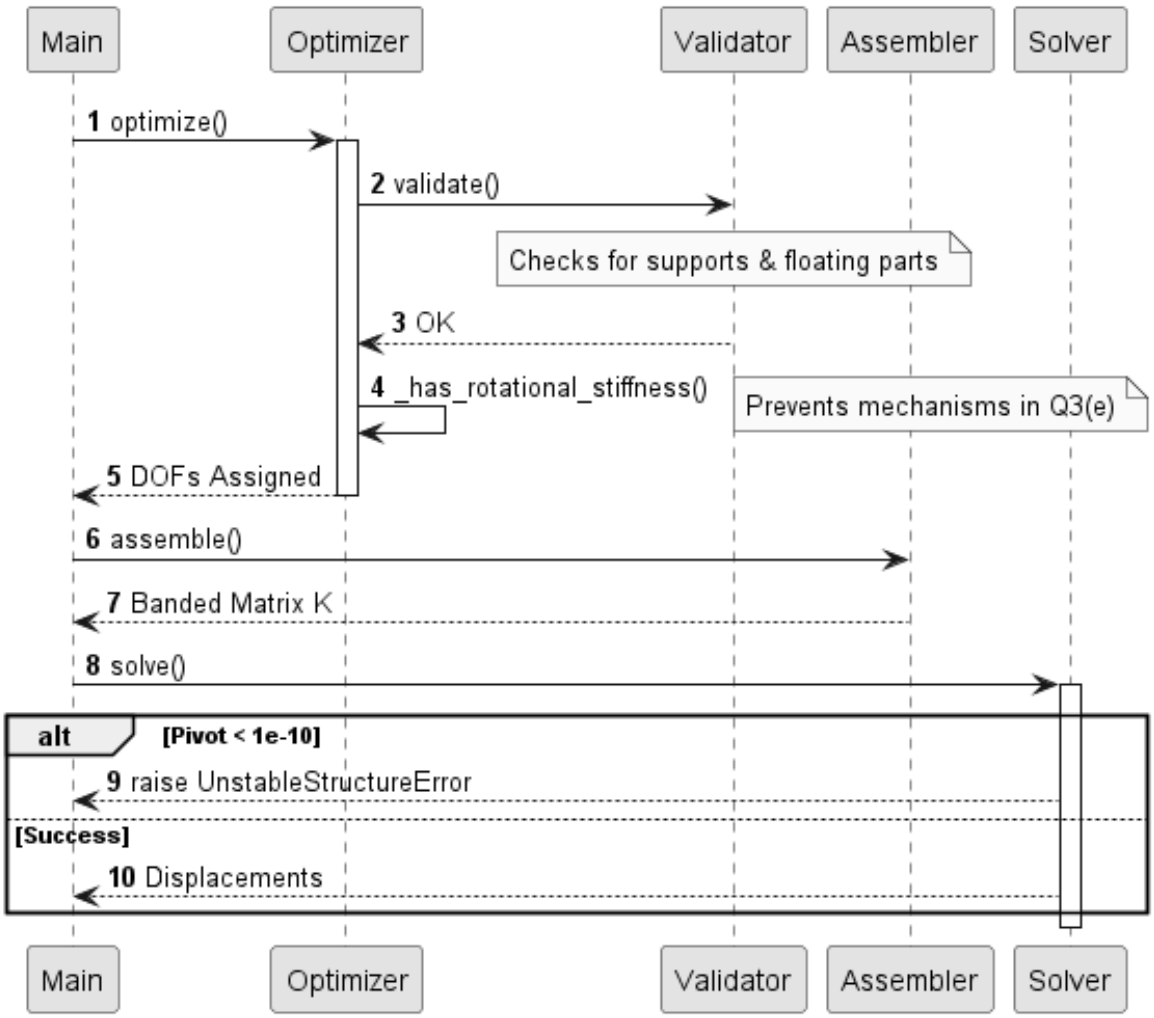

Figure A.1: UML Sequence Diagram showing inter-module message flow for a complete analysis run, including the `alt` block for `UnstableStructureError`.

A.2 SAP2000 Coordinate System Mapping

SAP2000 uses a 3D XZ-plane convention. The `_parse_sap2000()` method in `test_regression.py` applies the following mapping to align with the solver's 2D x-y convention:

Table A.1: SAP2000 to solver coordinate mapping.

SAP2000	**Solver**	**Notes**
U_1 (global X)	u_x	Direct map
U_3 (global Z)	u_y	Direct map
R_2 (rotation about Y)	$-r_z$	Sign inversion (CW vs CCW)

Table A.1 (continued)

SAP2000	**Solver**	**Notes**
F_1	F_x	Direct map
F_3	F_y	Direct map
M_2	$-M_z$	Sign inversion

A.3 Sign Convention References

Global axes: x horizontal (positive right), y vertical (positive up), r_z rotation positive CCW. Element local axes run from `node_i` to `node_j`. Frame DOF ordering is $[u_{x,i},\ u_{y,i},\ r_{z,i},\ u_{x,j},\ u_{y,j},\ r_{z,j}]^T$; truss elements use $[u_{x,i},\ u_{y,i},\ u_{x,j},\ u_{y,j}]^T$. All units are kN and metres.

All FEF computations in `element_physics.py` treat w and P as signed quantities (negative for downward loads). The FEF vector components are positive in the local element coordinate directions: $F_{y,i}$ and $F_{y,j}$ positive upward in local y; $M_{z,i}$ and $M_{z,j}$ positive CCW. FEFs are subtracted from the applied load vector during assembly: $\{F\}_{\text{net}} = \{P\}_{\text{nodal}} - \{FEF\}$.